



Session 09:

Session, Cookie, I18N



- 1. Tổng quan về Session & Cookie**
- 2. Làm việc với Session trong Spring MVC**
- 3. Quản lý Cookie trong Spring MVC**
- 4. Đa ngôn ngữ (i18n) trong Spring MVC**

1

Tổng quan về Session, Cookie

HTTP là giao thức Stateless

Để hiểu tại sao cần Session và Cookie, chúng ta phải nắm rõ bản chất của HTTP — giao thức nền tảng của mọi ứng dụng web.

Mỗi request là độc lập

Server xử lý từng request riêng biệt, không có liên kết nào giữa các request liên tiếp. Khi bạn gửi request thứ hai, server đã "quên" request thứ nhất.

Không có bộ nhớ phiên

HTTP không tự động lưu giữ thông tin người dùng giữa các lần kết nối. Server không biết ai đang truy cập hay họ đã làm gì trước đó.

Ưu điểm của Stateless

Đơn giản, dễ mở rộng (scale), phù hợp với kiến trúc phân tán. Mỗi server có thể xử lý bất kỳ request nào mà không cần chia sẻ trạng thái.

Vấn đề thực tế của Stateless

Tính Stateless của HTTP tạo ra nhiều thách thức trong thực tế khi xây dựng ứng dụng web yêu cầu trải nghiệm liên tục

Giỏ hàng bị mất

Người dùng thêm sản phẩm vào giỏ, chuyển sang trang khác và toàn bộ dữ liệu biến mất vì server không nhớ.

Đăng nhập không ghi nhớ

Mỗi lần chuyển trang, người dùng phải xác thực lại vì server không lưu trạng thái đăng nhập.

Dữ liệu form bị xóa

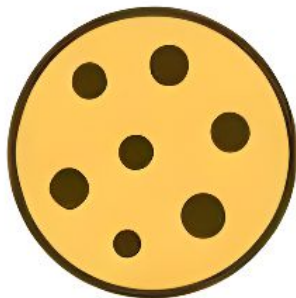
Trong form nhiều bước (wizard), dữ liệu đã nhập ở bước trước bị mất khi chuyển sang bước tiếp theo.

Kết luận: Cần một cơ chế để duy trì trạng thái người dùng xuyên suốt các request.

Giải pháp: Session & Cookie

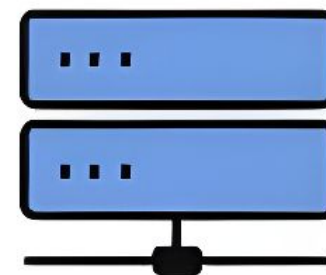
Để giải quyết vấn đề Stateless, Web cung cấp hai cơ chế bổ sung cho nhau: Session lưu dữ liệu trên Server và Cookie lưu dữ liệu trên Client.

COOKIE



VS

SESSION



Session là gì?

Session là cơ chế lưu trữ dữ liệu phía server, được gắn với từng người cụ thể thông qua một mã định danh duy nhất (Session ID)

1 Lưu trữ trên Server

Toàn bộ dữ liệu session nằm trên bộ nhớ server (RAM), không được gửi về phía client. Điều này đảm bảo tính bảo mật và toàn vẹn dữ liệu.

2 Gắn với từng người dùng

Mỗi người dùng kết nối sẽ được cấp một session ID duy nhất. Server dùng ID này để phân biệt và phục vụ đúng dữ liệu cho từng người.

3 Ứng dụng thực tế

Lưu thông tin đăng nhập (user đã xác thực), giỏ hàng trong phiên mua sắm, dữ liệu tạm thời giữa các bước của form nhiều bước.

Cookie là gì?

Cookie là những mẫu dữ liệu nhỏ dưới dạng văn bản, được server gửi về và lưu trữ trực tiếp trong trình duyệt của người dùng.

Lưu trữ tại trình duyệt

Cookie nằm trên máy người dùng, không tốn tài nguyên server. Mỗi trình duyệt quản lý cookie của riêng nó

Dữ liệu dạng text nhỏ

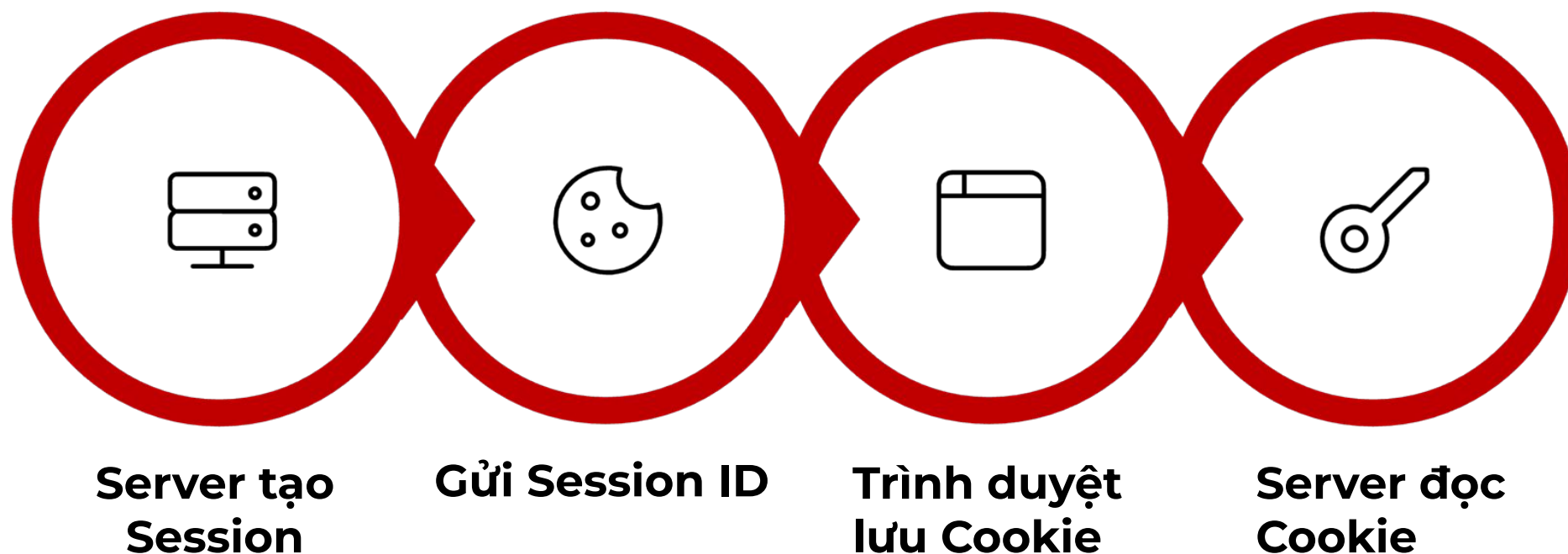
Cookie chỉ lưu được dữ liệu đơn giản dạng key-value. Giới hạn kích thước khoảng 4KB mỗi cookie.

Tự động gửi kèm request

Với mỗi HTTP request đến cùng domain, trình duyệt tự động đính kèm toàn bộ cookie liên quan vào header.

Cơ chế hoạt động: Session + Cookie

Session và Cookie phối hợp với nhau thông qua một cơ chế đơn giản nhưng hiệu quả để duy trì trạng thái đăng nhập của người dùng



So sánh Session và Cookie

Hiểu rõ sự khác biệt giữa Session và Cookie giúp bạn lựa chọn đúng công cụ cho từng tình huống cụ thể.

Tiêu chí	Session	Cookie
Nơi lưu trữ	Server (RAM)	Trình duyệt (Client)
Bảo mật	Cao - dữ liệu không lộ ra ngoài	Thấp hơn - có thể bị đọc/sửa
Hiệu năng	Tốn RAM server, cần quản lý	Nhẹ, không tốn tài nguyên server
Thời gian sống	Kết thúc khi đóng trình duyệt	Có thể cấu hình Max-Age tùy ý
Dung lượng	Không giới hạn cứng	Tối đa ~4KB mỗi cookie
Dùng cho	Đăng nhập, giỏ hàng, dữ liệu nhạy cảm	Cài đặt, ngôn ngữ, theme

Lưu ý bảo mật quan trọng

KHÔNG lưu vào Cookie

- Mật khẩu người dùng
- Access Token / Refresh Token
- Thông tin thẻ tín dụng
- Dữ liệu cá nhân nhạy cảm

NÊN lưu vào Cookie

- Session ID (được mã hóa)
- Cài đặt giao diện (theme, ngôn ngữ)
- Tùy chọn không nhạy cảm của người dùng
- Tracking ID (analytics)

Nguyên tắc cốt lõi

- Cookie là nơi công khai - bất kỳ ai cũng có thể đọc và sửa đổi
- Dữ liệu quan trọng luôn phải nằm trên server

2

Làm việc với Session trong Spring MVC

Session trong Spring MVC

Spring MVC cung cấp các annotation và interface tiện lợi để làm việc với Session mà không cần tương tác trực tiếp với Servlet API cấp thấp.

1

@SessionAttributes

- Đặt tại cấp **Class Controller**
- Tự động lưu các attribute được chỉ định từ **Model** vào **Session**.
- Phù hợp cho form nhiều bước (**wizard form**)

2

@SessionAttribute

- Đặt tại tham số của **method handler**
- Lấy dữ liệu từ **Session** và inject trực tiếp vào method - không cần gọi thủ công qua **HttpSession**

3

SessionStatus interface

- Inject vào method để kiểm soát vòng đời session.
- Gọi **setComplete()** để xóa session sau khi hoàn tất xử lý, tránh rò rỉ bộ nhớ

@SessionAttributes - Lưu dữ liệu vào Session

Annotation **@SessionAttributes** đặt ở cấp **class controller**, cho phép Spring tự động đồng bộ dữ liệu giữa **Model** và **Session**.

Cách hoạt động

Khi khai báo

@SessionAttributes("attribute"),
Spring tự động lưu attribute có tên đó từ Model vào Session sau mỗi request, truy cập được ở nhiều request khác

Trường hợp sử dụng

Form đặt hàng nhiều bước:

Bước 1: nhập thông tin cá nhân.

Bước 2: chọn sản phẩm

Bước 3: thanh toán

Dữ liệu mỗi bước được tự động lưu và truyền sang bước tiếp theo

Lưu ý quan trọng

Chỉ áp dụng trong phạm vi của **Controller** đó.

Dữ liệu sẽ tồn tại cho đến khi gọi **SessionStatus.setComplete()**

Không dùng cho **Restful API**

@SessionAttribute - Đọc dữ liệu từ Session

Annotation **@SessionAttribute** (không có 's') được dùng tại tham số của method, cho phép đọc trực tiếp giá trị từ Session một cách gọn gàng.

Đặc điểm

- Dùng tại tham số của method handler
- Spring tự động tìm, inject giá trị từ Session
- Không cần gọi session.getAttribute() một cách thủ công

So sánh cách viết

- Cách cũ (**Servlet API**): Phải inject **HttpSession**, gọi **getAttribute()**, ép kiểu thủ công - dài dòng, dễ lỗi
- Cách mới (**@SessionAttribute**): Chỉ cần thêm annotation vào tham số - **Spring** xử lý toàn bộ phần còn lại tự động, **type-safe**

SessionStatus - Xóa Session đúng lúc

SessionStatus là interface quan trọng giúp quản lý vòng đời của session, đảm bảo dữ liệu được dọn dẹp sau khi không còn cần thiết.

Mục đích sử dụng

Gọi **status.setComplete()** trong method xử lý cuối cùng của flow (ví dụ: sau khi **submit form**, để thông báo cho Spring xóa toàn bộ **session attributes**.

Hậu quả nếu quên

Session tích lũy dữ liệu không cần thiết theo thời gian, dẫn đến **Memory Leak**. Trên hệ thống nhiều người dùng, điều này làm cạn kiệt **RAM server** nhanh

Thời điểm gọi

Luôn gọi **setComplete()** sau khi hoàn thành tất form nhiều bước, sau khi đăng xuất hoặc khi người dùng hủy flow đang thực hiện

Lỗi thường gặp với Session

Nắm vững các lỗi phổ biến thường gặp với Session giúp bạn tránh những sự cố nghiêm trọng trong môi trường production.

1. Quên xóa Session

Không gọi **SessionStatus.setComplete()** sau khi hoàn tất. **Hậu quả:** dữ liệu tồn đọng, **Memory Leak** theo thời gian, **server** chậm dần

2. Lưu quá nhiều dữ liệu

Nhồi nhét **object** lớn, danh sách dài vào **session**. **Hậu quả:** tốn **RAM**, ảnh hưởng hiệu năng hệ thống khi nhiều người dùng đồng thời sử dụng.

3. Dùng Session trong API

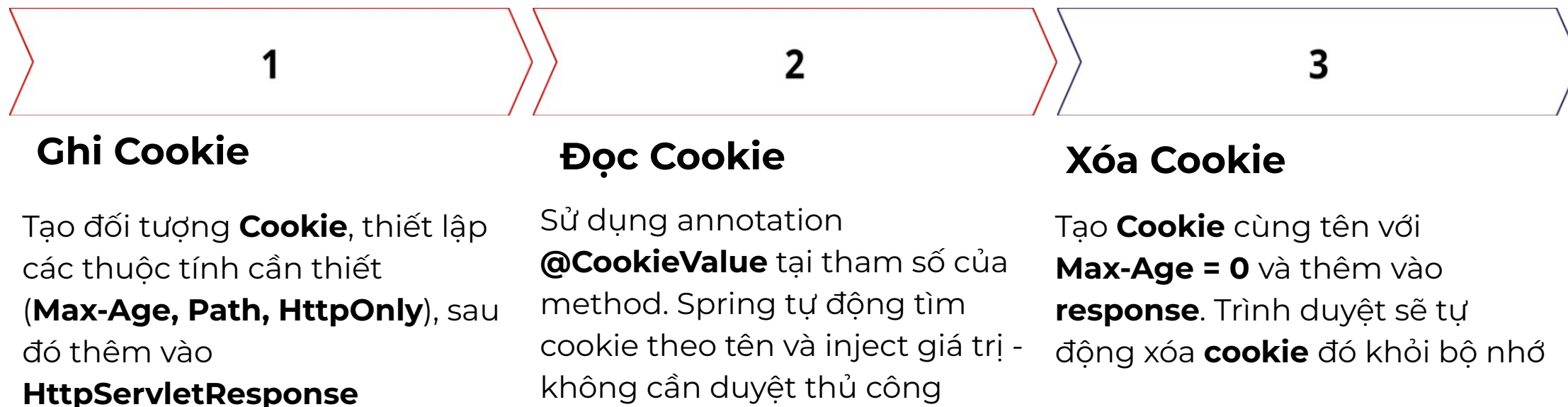
RESTful API theo nguyên tắc **Stateless** - không dùng **Session**. Thay vào đó hãy dùng **JWT** hoặc **OAuth2** để xác thực trong **RESTful API**

3

Quản lý Cookie trong Spring MVC

Cookie trong Spring MVC

Spring MVC cung cấp cách tiếp cận đơn giản và sạch sẽ để đọc và ghi **Cookie**, không cần thao tác thủ công với **Servlet API**.



@CookieValue - Đọc Cookie nhanh chóng

Annotation **@CookieValue** là cách hiện đại và thanh lịch để đọc giá trị trong **Spring MVC**, thay thế hoàn toàn cách duyệt thủ công.

Thuộc tính của @CookieValue

- **value**: Tên của cookie cần đọc
- **required**: Mặc định true - nếu cookie không tồn tại sẽ throw Exception
- **defaultValue**: Giá trị mặc định nếu cookie không tồn tại

Ứng dụng thực tế

- Đọc ngôn ngữ người dùng đã chọn
@CookieValue("lang")
- Đọc theme giao diện **@CookieValue("theme")**
- Đọc session ID: **@CookieValue("JSESSIONID")**

Thời gian sống của Cookie (Max-Age)

Thuộc tính **Max-Age** quyết định **cookie tồn tại bao lâu** và **được xóa như thế nào** - đây là cấu hình quan trọng nhất khi thiết kế **cookie**

Max-Age > 0

Cookie sống theo số giây được chỉ định. VD: **setMaxAge(3600)** -> cookie tồn tại 1 giờ. Vẫn còn sau khi đóng trình duyệt

Max-Age = 0

Cookie bị xóa ngay lập tức. Đây là cách chuẩn để **xóa một cookie, tạo cookie cùng tên** với **Max-Age = 0** và gửi về trình duyệt

Max-Age = -1

Session Cookie: cookie chỉ tồn tại trong phiên làm việc hiện tại. Khi người dùng đóng trình duyệt, cookie sẽ tự động bị xóa

Bảo mật Cookie: HttpOnly & Secure

Cấu hình đúng các thuộc tính bảo mật của cookie là bước không thể bỏ qua để bảo vệ ứng dụng khỏi các cuộc tấn công phổ biến

HttpOnly = true

Ngăn JS phía client đọc cookie thông qua **document.cookie**. Đây là biện pháp phòng thủ chủ yếu chống lại tấn công **XSS (Cross-Site Scripting)**. Nên bật cho mọi cookie chứa **Session ID**

Secure = true

Cookie chỉ được gửi qua kết nối **HTTPS**, không bao giờ qua **HTTP** thường. Bảo vệ **cookie** khỏi bị đánh cắp qua tấn công **Man-in-the-Middle (MITM)**. Luôn bật trong môi trường production

SameSite

Kiểm soát cookie có được gửi trong cross-site request hay không. Giá trị **Strict** hoặc **Lax** giúp chống tấn công **CSRF (Cross-Site Request Forgery)** hiệu quả.

4

Đa ngôn ngữ (i18n) trong Spring MVC

I18N là gì?

I18N là viết tắt của **Internationalization** - chữ 'I', 18 ký tự ở giữa, và chữ 'N'. Đây là quá trình thiết kế ứng dụng để có thể thích nghi với nhiều ngôn ngữ và văn hóa khác nhau mà không thay đổi code.

Vấn đề cần giải quyết

Không **hard-code** trực tiếp vào HTML hay Java. Thay vào đó dùng **key** để tham chiếu đến bản dịch phù hợp theo ngôn ngữ của người dùng.

Nguyên tắc hoạt động

Thay vì viết **“Xin chào”** trực tiếp, bạn viết key **“greeting”**. Hệ thống **I18N** sẽ load đúng bản dịch tương ứng. Ví dụ:
vi: “Xin chào”, en: “Hello”

Lợi ích

Thêm ngôn ngữ mới chỉ cần tạo file **properties** mới - không cần sửa code **Java** hay **template HTML**. Dễ bảo trì và mở rộng quy mô quốc tế

Nguyên lý I18N: Key thay vì Text

Thay vì nhúng text trực tiếp vào giao diện, I18N sử dụng hệ thống key-value để tách biệt nội dung ra khỏi code.

Cách sai - Hard-code text

- Nhúng trực tiếp “Hello” vào HTML template khi cần hỗ trợ tiếng anh
- Điều này vi phạm nguyên tắc tách biệt mối quan tâm

Cách đúng - Dùng key

Trong file **messages_vi.properties** ta khai báo: **greeting=Xin chào**

Trong file **messages_en.properties** ta khai báo: **greeting=Hello**

Trong **Thymeleaf**: **{greeting}**

Hệ thống tự động chọn bản dịch đúng theo ngôn ngữ hiện tại của người dùng

Cấu hình I18N trong Spring MVC

Spring MVC cung cấp ba thành phần cốt lõi phối hợp với nhau để tạo nên hệ thống đa ngôn ngữ hoàn chỉnh



File Properties - Kho lưu bản dịch

Các bản dịch được lưu trong các file `.properties` riêng biệt cho từng ngôn ngữ, đặt trong thư mục `resources` của project.

messages.properties

File mặc định — được dùng khi không tìm thấy file ngôn ngữ phù hợp. Thường chứa bản tiếng Anh hoặc ngôn ngữ mặc định của ứng dụng.

messages_vi.properties

File bản dịch tiếng Việt. Được load khi Locale của người dùng là `vi`. Ví dụ: `login.title=Đăng nhập`, `nav.home=Trang chủ`.

messages_en.properties

File bản dịch tiếng Anh. Được load khi Locale của người dùng là `en`. Ví dụ: `login.title=Login`, `nav.home=Home`.

Cấu hình `MessageSource` với `setBasename("classpath:messages")` để Spring tự động tìm đúng file theo pattern `messages_[locale].properties`.

Chuyển ngôn ngữ theo URL

LocaleChangeInterceptor cho phép người dùng chuyển ngôn ngữ bằng cách thêm tham số vào URL — đơn giản và thân thiện với bookmark.

1

?lang=en

Chuyển sang tiếng Anh. URL ví dụ: `/home?lang=en`

2

?lang=vi

Chuyển sang tiếng Việt. URL ví dụ: `/home?lang=vi`

3

Tùy chỉnh param

Tên tham số có thể thay đổi: `setParamName("language")`

Interceptor này phải được đăng ký trong **WebMvcConfigurer** thông qua **addInterceptors()**. Khi nhận request có tham số **lang**, Spring cập nhật Locale và lưu vào Cookie hoặc Session tùy theo cấu hình **LocaleResolver**.

Cookie vs Session cho I18N

Khi lưu ngôn ngữ người dùng, việc chọn đúng cơ chế lưu trữ ảnh hưởng trực tiếp đến trải nghiệm người dùng

SessionLocaleResolver

- Ngôn ngữ mất khi đóng trình duyệt
- Người dùng phải chọn lại mỗi lần vào
- Phù hợp cho ứng dụng yêu cầu đăng nhập
- Không cần lưu trữ lâu dài

CookieLocaleResolver -> khuyến nghị

- Ngôn ngữ được nhớ ngay cả khi đóng trình duyệt
- Người dùng quay lại vẫn thấy ngôn ngữ đã chọn
- Cấu hình Max-Age để lưu lâu dài (ví dụ: 30 ngày)
- Phù hợp cho hầu hết ứng dụng web thực tế

Best Practice: Dùng `CookieLocaleResolver` cho **I18N**. Đây là lựa chọn tốt hơn vì tạo ra trải nghiệm nhất quán cho người dùng quay lại.

- ❑ **Session, Cookie và I18N tạo nên ứng dụng Spring MVC chuyên nghiệp, bảo mật, thân thiện với người dùng toàn cầu**
- ❑ **Session dùng cho dữ liệu nhạy cảm, đăng nhập, luôn gọi `setComplete()` sau khi xong, không dùng trong RESTful API**
- ❑ **Cookie dùng cho setting lâu dài, ngôn ngữ, theme, luôn bật `HttpOnly` và `Secure` và không lưu dữ liệu nhạy cảm**
- ❑ **I18N giúp hiện thực tính năng đa ngôn ngữ trong ứng dụng**
- ❑ **I18N không hard-code text trong HTML, dùng `CookieLocaleResolver`, tổ chức Key rõ ràng theo module**



KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN